

Reviewer Pack — Total-Influence Defect of Clause-Indicator Bounds Tree-Resol...

Entry 633d44f5ae88 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-27 21:18:38 UTC

Total-Influence Defect of Clause-Indicator Bounds Tree-Resolution Depth

Entry ID: 633d44f5ae88 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-27 21:18:38 UTC

1. Conjecture

Field A (mathematical branch): Fourier analysis of Boolean functions: total influence $I(g) = \sum_i \text{Inf}_i(g)$ and the KKL/Friedgut spectral concentration of the clause-indicator function $g_F(x) = \bigwedge C \text{ (clause } C \text{ is satisfied by } x\text{)}$, restricted to its 0-set (the set of UNSAT assignments) — specifically the defect $\Delta(F) = \log_2(\#\text{UNSAT}(F)) - I(g_F) \cdot n / (2 \cdot \#\text{UNSAT}(F))$, a low-cost spectral statistic **Field B** (complexity object): Tree-like resolution refutation depth $d_T(F \vdash \perp)$ of small unsatisfiable 3-CNFs, equivalently DPLL search-tree height on F

Statement:

For every unsatisfiable 3-CNF F on $n \leq 20$ variables with $m \leq 5n$ clauses, the tree-resolution depth satisfies $d_T(F \vdash \perp) \geq \lceil \Delta(F) \rceil$, where $\Delta(F) := \log_2(\max(1, \#\text{UNSAT}(F))) - (n \cdot I(g_F)) / (2 \cdot \max(1, \#\text{UNSAT}(F)))$ and $I(g_F) = \sum_i \Pr_x[g_F(x) \neq g_F(x \oplus e_i)]$ is computed by direct enumeration of the 2^n cube. Equivalently, the average pivot-influence of g_F lower bounds the DPLL height by a logarithmic-counting correction term.

Rationale (proposer's reasoning):

KKL-style total influence quantifies how 'spread' the UNSAT region is across coordinate flips; a DPLL tree of depth d partitions $\{0,1\}^n$ into $\leq 2^d$ leaves, each a subcube on which g_F is constant, so leaves with many UNSAT points must have small boundary, forcing $I(g_F)$ small unless d is large. The defect $\Delta(F)$ makes this trade-off quantitative without invoking switching lemmas, and—unlike entropy-of-leaves arguments already used in proof complexity—it is purely a Walsh-spectral quantity of g_F , an angle largely unexplored for resolution depth.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 27b184e2a3ea05c4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ~ 80 unsatisfiable 3-CNFs per $n \in \{8, 10, 12, 14, 16\}$ (≈ 400 instances total, fixed seeds 1..80), with #UNSAT and $I(g_F)$ by 2^n enumeration and d_T by memoized minimax DPLL: SUPPORT iff per- n support fraction $\Pr[d_T \geq \lceil \Delta(F) \rceil] \geq 0.95$ for every n and overall mean slack $(d_T - \lceil \Delta \rceil) > 0$; FALSIFY on any single instance with $d_T < \lceil \Delta(F) \rceil$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.92	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - total influence Boolean function resolution proof complexity DPLL - Fourier analysis unsatisfiable CNF tree resolution lower bound - clause indicator function spectral KKL DPLL search tree depth

Top relevant hits considered: - [<http://arxiv.org/abs/cs/0209032v3>] Complexity Results on DPLL and Resolution - [<http://arxiv.org/abs/2003.09703v1>] Variance function of boolean additive convolution - [<http://arxiv.org/abs/1407.6169v3>] Multiplicative Complexity of Vector Valued Boolean Functions - [<http://arxiv.org/abs/0806.1148v2>] Unsatisfiable CNF Formulas need many Conflicts - [<http://arxiv.org/abs/0905.1587v5>] Unsatisfiable Linear CNF Formulas Are Large and Complex - [<http://arxiv.org/abs/2511.07739v3>] A Lower Bound for the Fourier Entropy of Boolean Functions on the Biased Hypercube - [<http://arxiv.org/abs/1208.6015v1>] The spectral function of a first order elliptic system - [<http://arxiv.org/abs/1009.0997v3>] Spectral asymptotics for Robin problems with a discontinuous coefficient

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import sys
from collections import defaultdict

def dpll(clauses, assignment):
    if not clauses:
        return True
    var = next(var for var in range(len(assignment)) if assignment[var] is None)
    for val in [True, False]:
        new_assignment = assignment[:]
        new_assignment[var] = val
        new_clauses = [c for c in clauses if not any(l == -var or l == var for l in c)]
        if dpll(new_clauses, new_assignment):
            return True
```

```

    return False

def is_unsat(clauses):
    assignment = [None] * len(clauses)
    return not dpll(clauses, assignment)

def count_unsat(clauses):
    n = len(clauses[0])
    count = 0
    for i in range(1 << n):
        assignment = [(i >> j) & 1 for j in range(n)]
        if is_unsat([(l + (n if l < 0 else -n) for l in c) for c in clauses]):
            count += 1
    return count

def indicator_flip(clauses, assignment, var):
    new_assignment = assignment[:]
    new_assignment[var] = not new_assignment[var]
    return is_unsat([(l + (n if l < 0 else -n) for l in c) for c in clauses])

def compute_I(g_F, n):
    I = 0
    for i in range(n):
        count = sum(indicator_flip(clauses, [None] * n, i) for _ in range(1 << (n - 1)))
        I += abs(count / (2 ** (n - 1)) - 0.5)
    return I

def compute_delta(F):
    n = len(F[0])
    m = len(F)
    unsat_count = count_unsat(F)
    if unsat_count == 0:
        return float('inf')
    delta = math.log2(unsat_count) - (n * compute_I(F, n)) / (2 * unsat_count)
    return max(0, delta)

def memoized_minimax(clauses, assignment, depth=0):
    if not clauses:
        return 0
    var = next(var for var in range(len(assignment)) if assignment[var] is None)
    best = float('inf')
    for val in [True, False]:
        new_assignment = assignment[:]
        new_assignment[var] = val
        new_clauses = [c for c in clauses if not any(l == -var or l == var for l in c)]
        best = min(best, memoized_minimax(new_clauses, new_assignment, depth + 1))
    return best

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16]
    results = []

    for n in n_values:
        m = max(4 * n // 3, 1)
        unsat_count = 0

```

```

support_count = 0

while unsat_count < 80:
    clauses = []
    for _ in range(m):
        clause = [random.choice([-i-1, i+1] for i in range(n)) for _ in range(3)]
        if is_unsat(clause):
            clauses.append(clause)
    if len(clauses) == m and not dpll(clauses, [None] * n):
        unsat_count += 1
        g_F = [[l + (n if l < 0 else -n) for l in c] for c in clauses]
        delta = compute_delta(g_F)
        d_T = memoized_minimax(clauses, [None] * n)
        if d_T >= math.ceil(delta):
            support_count += 1
            results.append((d_T, math.ceil(delta)))

support_fraction = support_count / len(results)
mean_slack = sum(d_T - math.ceil(delta) for d_T, delta in results) / len(results)
conjecture_holds = support_fraction >= 0.95 and mean_slack > 0

return {
    "metric_name": "slack",
    "metric_value": mean_slack,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": "" if conjecture_holds else f"support_fraction={support_fraction}, n
}

if __name__ == "__main__":
    seeds = [int(s) for s in sys.argv[1:]] or [11, 23, 37, 53, 71]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_slack = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_slack} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={result['counterexample']}\n")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_82aca615.py", line 116, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_82aca615.py", line 88, in run_trial
    clause = [random.choice([-i-1, i+1] for i in range(n)) for _ in range(3)]
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/usr/lib/python3.12/random.py", line 346, in choice
    if not len(seq):
            ^^^^^^^
TypeError: object of type 'generator' has no len()

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a TypeError due to a bug in clause construction (random.choice was passed a generator instead of a list), so no instances were ever evaluated and the pre-registered support condition cannot be assessed. | next: Fix the clause sampler (e.g., random.choice([-i-1, i+1]) for a chosen i, or wrap the generator in list(...)) and rerun the 80-instance-per-n sweep for n in {8,10,12,14,16}.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	19552
2	preregistration	claude_max	opus	0	0	6314
3	novelty	claude_max	opus	0	0	3944
4	novelty	claude_max	opus	0	0	8547
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17264
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13916
7	judge	claude_max	opus	0	0	4761

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 74299 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in research/audit/633d44f5ae88.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/633d44f5ae88.jsonl for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/633d44f5ae88.tar.gz` (if generated)